Task Scheduling Explanation

Absolutely. Your code is using a **max heap + cycle of `n + 1` slots**. The important idea is that after executing a task, the same task cannot execute again for the next `n` positions.

Let's understand **every important line**, then do a complete dry run with:

```
tasks = A A A B B B
n = 2
```

Expected answer:

```
8
```

1. What does `n = 2` actually mean?

If:

```
n = 2
```

then after executing a task, we need **2 other time slots before the same task can execute again**.

For example:

```
A _ _ A
```

So the distance between two `A`s must be:

```
n + 1 = 3
```

That's why your code processes tasks in groups of:

```
C++
n + 1
```

2. Frequency array

```
C++
vector<int>mpp(26, 0);
```

There are 26 uppercase English letters:

```
A B C D ... Z
```

Initially:

```
A: 0
B: 0
C: 0
...
Z: 0
```

Then:

```
C++
for(char &ch : tasks){
    mpp[ch - 'A']++;
}
```

Why `ch - 'A'`?

Characters have ASCII values.

```
'A' - 'A' = 0
'B' - 'A' = 1
'C' - 'A' = 2
```

Therefore:

```
A → index 0
B → index 1
C → index 2
```

For:

```
A A A B B B
```

we get:

```
mpp[0] = 3   // A
mpp[1] = 3   // B
```

Everything else is **0**.

---

3. Create max heap

```
C++
priority_queue<int> pq;
```

By default, C++ **priority_queue** is a **max heap**.

So the largest frequency stays at the top.

Then:

```
C++
for(int i = 0; i < 26; i++){
    if(mpp[i] > 0){
        pq.push(mpp[i]);
    }
}
```

We push the frequencies.

For our example:

```
A = 3
B = 3
```

So:

```
pq = {3, 3}
```

Conceptually:

```
      3   ← largest
     / \
    3
```

We don't actually store the task letter, only its frequency.

---

4. Start time

```
C++
int time = 0;
```

Initially:

```
time = 0
```

5. Main loop

```
C++
while(!pq.empty()){
```

As long as there are tasks remaining, keep scheduling.

Initially:

```
pq = {3,3}
```

so we enter the loop.

---

6. Temporary vector

```
C++
vector<int> temp;
```

This is VERY important.

Suppose we execute:

```
A
B
```

Their frequencies become:

```
A: 2
B: 2
```

But we **cannot immediately push them back into the heap**.

Why?

Because if we push `A` back immediately, we could pick `A` again in the same cycle.

That would violate the cooldown.

So:

```
pq
 ↓
take tasks
 ↓
temp
 ↓
after cycle
 ↓
put back into pq
```

`temp` temporarily holds the tasks we have already used in the current cycle.

---

7. The cycle

```C++
for(int i = 1; i <= n + 1; i++){
```

Since:

```
n = 2
```

we get:

```
n + 1 = 3
```

So:

```C++
for(int i = 1; i <= 3; i++)
```

We try to fill **3 positions**.

Think of it as:

```
[A][B][A]
```

or:

```
[A][B][idle]
```

depending on how many different tasks remain.

---

8. Check if heap has tasks

```C++
if(!pq.empty()){
```

If there is a task available, execute it.

Otherwise, we don't execute anything.

That empty position effectively becomes an **idle time**.

---

9. Take highest frequency task

```
C++
int freq = pq.top();
```

Take the task with the highest remaining frequency.

Initially:

```
pq = {3,3}
```

So:

```
freq = 3
```

10. Remove it from heap

```
C++
pq.pop();
```

Now that task is temporarily removed.

Before:

```
pq = {3,3}
```

After popping:

```
pq = {3}
```

11. Execute one occurrence

```
C++
freq--;
```

We executed the task once.

So:

```
frequency = 3
```

becomes:

```
frequency = 2
```

For example:

```
A A A
```

If we execute one **`A`**:

```
A A
```

remain.

---

12. Store in temp

```
C++
temp.push_back(freq);
```

We put the remaining frequency into **`temp`**.

So:

```
temp = {2}
```

This means:

> We executed one occurrence of this task, and 2 occurrences remain.

---

13. Continue the cycle

Our cycle has 3 slots.

First slot

Take:

```
A(3)
```

Execute it:

```
A(2 remaining)
```

```
temp = {2}
pq = {3}
```

Second slot

Take:

```
B(3)
```

Execute it:

```
B(2 remaining)
```

```
temp = {2,2}
pq = {}
```

Third slot

Now:

```
C++
if(!pq.empty())
```

is false.

So nothing happens.

This becomes an idle slot:

```
A B idle
```

14. Put tasks back into heap

After the cycle:

```
C++
for(int &f : temp){
    if(f > 0){
        pq.push(f);
    }
}
```

Our:

```
temp = {2,2}
```

Both are greater than zero.

So:

```
pq = {2,2}
```

Now we can use them in the **next cycle**.

---

15. Why don't we push immediately?

This is one of the most important parts of the algorithm.

Suppose we did:

```
C++
pq.push(freq);
```

immediately after executing `A`.

Then:

```
pq = {2,3}
```

and we might pick:

```
A
```

again.

We could get:

```
A A ...
```

which violates:

```
A _ _ A
```

So we wait until the current **n+1** cycle is finished.

---

16. Calculate time

Now:

```C++
if(pq.empty()){
    time += temp.size();
}
else{
    time += n + 1;
}
```

There are two cases.

Case 1: Heap is NOT empty

If tasks are still remaining:

```C++
time += n + 1;
```

Why?

Because we need the complete cycle.

For:

```
n = 2
```

the cycle has:

```
3 positions
```

For example:

```
A B idle
```

So:

```
time += 3
```

Case 2: Heap IS empty

This is subtle.

Suppose the last cycle is:

```
A B
```

There is no reason to add the final idle slot.

We don't need:

```
A B idle
```

because there are no more tasks afterward.

So:

```C++
time += temp.size();
```

If we executed 2 tasks:

```
time += 2
```

instead of:

```
time += 3
```

Complete dry run

Input:

```
A A A B B B
n = 2
```

Frequency:

```
A = 3
B = 3
```

Heap:

```
{3,3}
```

Cycle 1

Cycle size:

```
n + 1 = 3
```

Slot 1

Take A:

```
A: 3 → 2
```

```
temp = {2}
```

Slot 2

Take B:

```
B: 3 → 2
```

```
temp = {2,2}
```

Slot 3

Heap empty:

```
idle
```

So current schedule:

```
A B idle
```

Then put **`temp`** back:

```
pq = {2,2}
```

Heap isn't empty, so:

```
time += 3
```

Now:

```
time = 3
```

Cycle 2

Heap:

```
{2,2}
```

Slot 1

Take A:

```
A: 2 → 1
```

Slot 2

Take B:

```
B: 2 → 1
```

Slot 3

Heap empty → idle.

Schedule:

```
A B idle
```

Then:

```
temp = {1,1}
```

Put back:

```
pq = {1,1}
```

Heap isn't empty:

```
time += 3
```

Now:

```
time = 6
```

Cycle 3

Heap:

```
{1,1}
```

Slot 1

Take A:

```
1 → 0
```

Slot 2

Take B:

```
1 → 0
```

Now:

```
temp = {0,0}
```

Heap:

```
{}
```

There are no more tasks.

Schedule:

```
A B
```

NOT

```
A B idle
```

Therefore:

```
C++
time += temp.size();
```

`temp.size()` is **2**.

So:

```
time = 6 + 2
     = 8
```

Final schedule:

```
A B _ A B _ A B
```

Therefore:

```
8
```

Visualizing the whole algorithm

```
Initial frequencies

A = 3
B = 3

        MAX HEAP
       ┌─────┐
       │  3  │ ← A
       │  3  │ ← B
       └─────┘
          │
          ▼
      Cycle size = 3
        (n + 1)
          │
     ┌────┼────┐
     ▼    ▼    ▼
     A    B   IDLE
     │    │
    3→2  3→2
     │    │
     └──┬─┘
        ▼
      temp
     [2, 2]
        │
        ▼
    push back
        │
        ▼
      HEAP
     [2, 2]
        │
        ▼
     Next cycle
```

Eventually:

```
Cycle 1:  A B _
Cycle 2:  A B _
Cycle 3:  A B
          ↑
       no idle
```

Total:

```
3 + 3 + 2 = 8
```

Your entire code in conceptual form

Your code is basically doing this:

```
1. Count how many times every task occurs
                ↓
2. Put frequencies into max heap
                ↓
3. Take the most frequent tasks first
                ↓
4. Execute at most n+1 different tasks
                ↓
5. Decrease their frequencies
                ↓
6. Temporarily store them in temp
                ↓
7. After the cycle, put remaining tasks back
                ↓
8. If tasks remain → count complete n+1 cycle
                ↓
9. If no tasks remain → count only actual tasks
                ↓
10. Repeat
```

The most important three ideas to remember

1. Why max heap?

```
C++
priority_queue<int> pq;
```

Because we always want to execute the task having the **highest remaining frequency** first.

2. Why `n + 1`?

Because if cooldown is `n`, the same task needs:

```
task + n spaces = n + 1 positions
```

For `n = 2`:

```
A _ _ A
↑     ↑
distance = 3 = n+1
```

3. Why `temp`?

Because a task used in the current cycle must **not immediately become available again**.

```
PQ → execute → TEMP
               ↓
       finish cycle
               ↓
         PQ again
```

That's the core trick that makes your heap solution work.